



Arquitetura de Software para Monitoramento e Prevenção de Incêndios Florestais

Baseada em IoT, Apache Kafka e Spark Structured Streaming

ARTIGO CIENTÍFICO

PAULO CÉSAR PEREIRA

O Problema: Por Que Sistemas Tradicionais Falham?

Ameaça Crescente

Incêndios florestais causam perdas irreparáveis de biodiversidade, emissões massivas de gases do efeito estufa e severos impactos socioeconômicos globais.

Limitações do Monitoramento Atual

Sensoriamento remoto por satélites e patrulhamento visual humano falham em prover a granularidade temporal necessária para supressão de focos em estágios iniciais – limitados por órbita, cobertura de nuvens e atrasos severos na transmissão de alertas.

O Desafio do Big Data Georreferenciado

Redes de sensores distribuídos geram fluxos em altíssima frequência. Sistemas monolíticos ou em batch são inadequados: a latência inviabiliza alertas em tempo hábil.

Visão Geral da Arquitetura Proposta

Uma solução de ponta a ponta para detecção precoce de anomalias climáticas indicativas de focos de incêndio, orquestrada via Docker.

Camada 1 — Edge AI (Captação)

20 nós simulados em Python emitem payloads JSON a cada 5s com temperatura, umidade e CO2. Classificação binária local (0 = normal, 1 = alerta).

Camada 2 — Apache Kafka (Ingestão)

Middleware distribuído de alto desempenho. Produtor injeta dados no tópico `dados-sensores`. Buffer resiliente com desacoplamento temporal e espacial.

Camada 3 — Spark Structured Streaming (Processamento)

Consume mensagens via abstração de tabela infinita. Calcula o Índice de Risco de Incêndio (IRI) em micro-batches com semântica `exactly-once`.

Camada 4 — PostgreSQL (Persistência)

Data Sink com armazenamento JSONB. Unifica leituras brutas e IRI calculado. Pronto para consumo via APIs REST.

As quatro camadas são modulares e fracamente acopladas, garantindo escalabilidade horizontal, tolerância a falhas e baixa latência no processamento de micro-batches.

Camada 1: Captação na Borda (Edge AI)

20 Nós Simulados

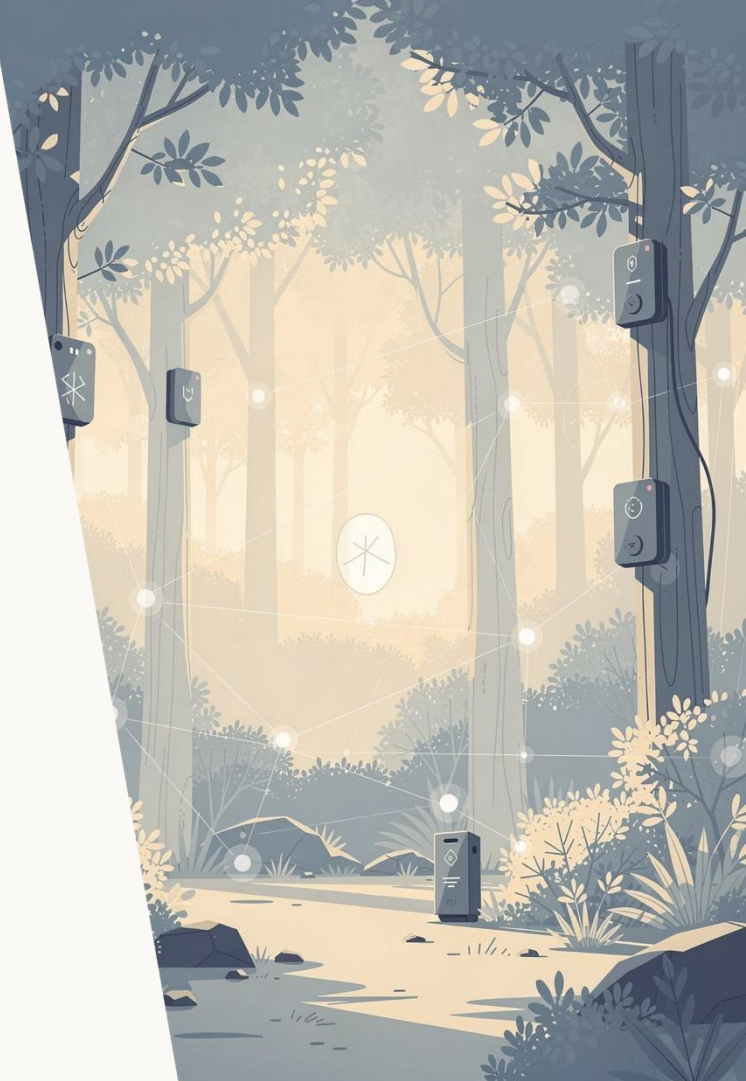
Sensores estocásticos independentes programados em Python, emitindo payloads JSON a cada **5 segundos**.

Variáveis Coletadas

Cada payload contém **sensor_id**, **timestamp**, **temperatura**, **umidade** e **CO2**.

Classificação Binária

Algoritmo de Edge AI classifica o estado ecológico: **0** = normalidade, **1** = alerta iminente.



Camadas 2 e 3: Kafka + Spark Structured Streaming

Apache Kafka — Ingestão Distribuída

Atua como middleware de alto desempenho e buffer resiliente. O produtor injeta dados no tópico **dados-sensores**, garantindo desacoplamento temporal e espacial e prevenindo perdas por saturação de rede.



Spark Structured Streaming — Processamento

Consome mensagens do Kafka via abstração de **tabela infinita** atualizada por micro-batches. Garante semântica **exactly-once**, prevenindo duplicidade ou omissão de registros críticos de incêndio.

O Kafka protege a integridade dos dados meteorológicos contra instabilidades de rede; o Spark garante fidelidade cronológica via funções nativas SQL como `date_trunc`.

O Índice de Risco de Incêndio (IRI)

Métrica analítica ponderada calculada pelo Spark para quantificar a severidade climática instantânea. Se a Edge AI indicar **Estado 1 (Alerta)**, o IRI é elevado imediatamente ao teto de **100,0**.

1

Delta de Temperatura (ΔT) $\times 4,0$

Temperatura acima de **30°C** ativa o fator de maior peso. Cada grau acima do limite multiplica o risco por 4, refletindo o alto impacto térmico na propagação do fogo.

2

Delta de Umidade (ΔU) $\times 1,5$

Umidade abaixo de **40%** penaliza o sistema. Cada ponto percentual abaixo do limite crítico é multiplicado por 1,5, pois ar seco acelera a combustão.

3

Delta de CO2 (ΔCO_2) $\times 0,1$

CO2 acima de **450 ppm** indica combustão. O multiplicador menor (0,1) atua como indexador de escala longa, pois em queima o nível sobe para milhares de ppm rapidamente.

Camada 4: Persistência em PostgreSQL (Data Sink)

Armazenamento JSONB

O Spark injeta os objetos JSON via conector JDBC na tabela **historico_sensores**, utilizando colunas do tipo **JSONB** (JSON Binário). Essa estratégia elimina a dependência de esquemas relacionais fixos e transforma o banco em um Data Sink genérico e flexível.

Interoperabilidade

A estrutura unifica leituras físicas brutas e o IRI calculado, pronta para consumo assíncrono via **APIs REST** ou barramentos de microserviços externos.

Benefícios Comprovados

- Timestamps precisos e ordenação estrita
- Alta confiabilidade e consistência referencial
- Índices JSONB otimizados para consultas de alta concorrência
- Repositório centralizado de telemetria ambiental

Data Sink (coletor de dados) é o destino final em um fluxo ou pipeline de dados.

Resultados: Desempenho e Escalabilidade

Testes de estresse com carga progressiva de 20 a 5.000 nós simultâneos validaram a robustez da arquitetura.

40ms

Latência até 1.000 nós

Tempo de resposta do broker Kafka mantido abaixo de 40 ms (39,0 ms) para até 1.000 dispositivos simultâneos.

607ms

Latência com 5.000 nós

Aumento exponencial ao superar 2.000 dispositivos, atingindo 607,2 ms com 5.000 sensores concorrentes.

1.000

Mensagens por segundo

Taxa contínua de transferência sustentada com 5.000 nós, mantendo o cluster operacional e resiliente (latência máx. 589 ms).

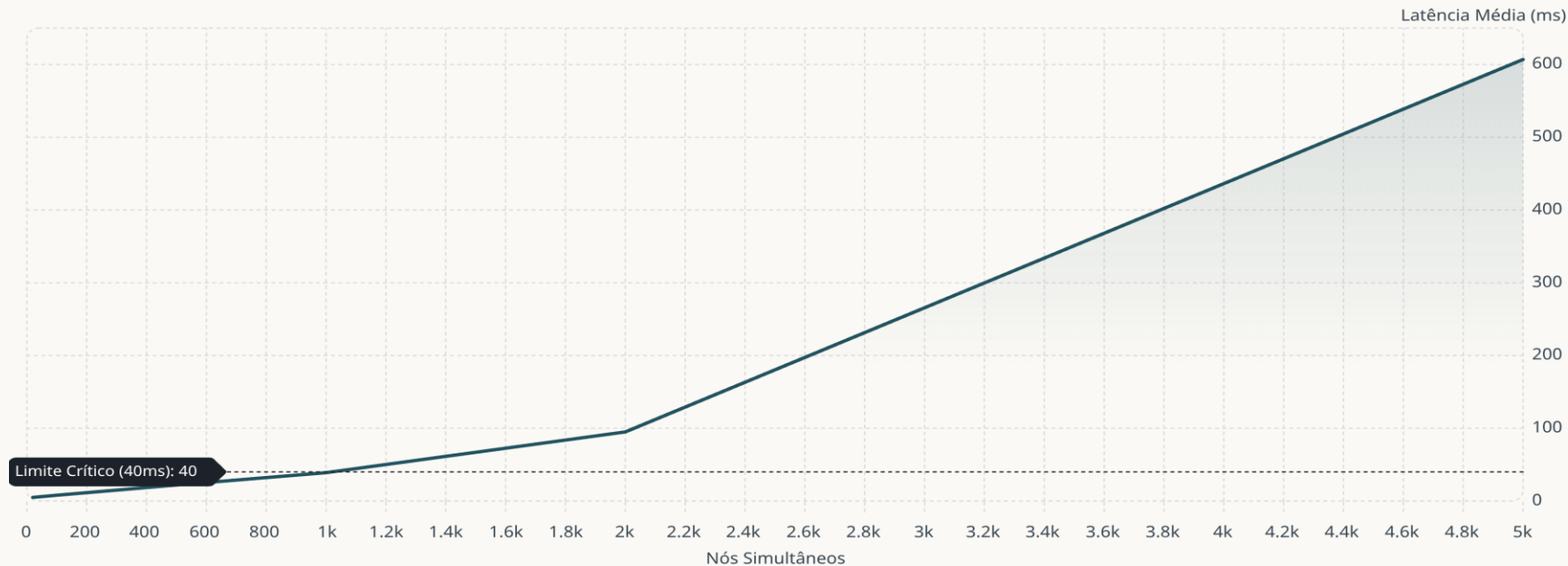
100.0

IRI máximo propagado

Ao detectar Estado 1 na borda, o Spark elevou o IRI ao teto imediatamente, sem backpressure ou picos de memória RAM.

Latência vs. Escala: Comportamento do Cluster

O gráfico abaixo ilustra o crescimento exponencial da latência ao ultrapassar 2.000 nós simultâneos, evidenciando o ponto crítico de saturação da arquitetura.



⚠ Acima de 1.000 nós, a latência supera o limite crítico de 40ms. Para escala superior a 500 sensores, recomenda-se migração para HDFS ou Amazon S3 como Data Sink.

Discussão: Lições e Limitações

☒ O Que Funcionou

- Kafka como buffer distribuído impediu perda de pacotes mesmo durante agregações complexas do Spark
- Semântica **exactly-once** eliminou sobreposição de registros em janelas concorrentes
- Conversão da lógica da Edge AI para inteiros binários (0/1) reduziu overhead de serialização
- JSONB indexado mitigou o overhead de parsing de colunas relacionais puras

☒ Gargalo Identificado

Acima de **400 nós concorrentes**, o gargalo físico de I/O JDBC persistiu devido à natureza síncrona das conexões de banco de dados.

☒ Solução Recomendada

Para escalas superiores a 500 sensores, a evolução para sistemas de arquivos distribuídos como **HDFS** ou Object Storages como **Amazon S3** é necessária para eliminar o estrangulamento relacional.

Conclusão e Trabalhos Futuros

A arquitetura validou o uso de tecnologias **open-source** orquestradas via containerização para infraestruturas de alta interoperabilidade e baixo custo aplicadas à preservação ecológica.



Hardware Real (ESP32)

Substituir o simulador Python por nós físicos com sensores MQ/DHT operando via protocolo **LoRaWAN**.



API REST com FastAPI

Expor os dados via API para consumo por **aplicativos móveis** de brigadas de incêndio e agências ambientais.



IA Preditiva (Random Forest)

Incorporar modelos de Machine Learning no fluxo do Spark para **predição proativa** de focos, além da detecção reativa.



Obrigado!

Perguntas?

Estamos à disposição para esclarecer qualquer dúvida sobre nossa arquitetura e resultados.

A codificação utilizada está disponível no GitHub: https://github.com/pclinux-48/topics_computer_systems.

Linkedin: <https://www.linkedin.com/in/paulocesarper/>

GitHub: <https://github.com/pclinux-48>

Referências

- AL-FUQAHA, Ala et al. Internet of things: A survey on enabling technologies, protocols, and applications. IEEE Communications Surveys & Tutorials, v. 17, n. 4, p. 2347-2376, 2015.
- ARMBRUST, Michael et al. Structured streaming: A declarative api for real-time applications in apache spark. In: Proceedings of the 2018 International Conference on Management of Data. 2018. p. 601-613.
- BUYA, Rajkumar; DASTJERDI, Amir Vahid (Ed.). Internet of Things: Principles and paradigms. Elsevier, 2016.
- GAMA, Joao. Knowledge Discovery from Data Streams. CRC Press, 2010.
- KARAU, Holden et al. Learning Spark: Lightning-Fast Big Data Analysis. O'Reilly Media, Inc., 2015.
- KLEPPMANN, Martin. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media, Inc., 2017.
- KREPS, Jay; NARKHEDE, Neha; RAO, Jun. Kafka: A distributed messaging system for log processing. In: Proceedings of the NetDB. 2011. p. 1-7.
- SHAFIUR REHMAN, Muhammad et al. Edge AI: On-device intelligence for next-generation IoT applications. IEEE Access, v. 7, p. 147230-147255, 2019.
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Database System Concepts. 7. ed. McGraw-Hill, 2020.
- SOWMYA, V.; PONMANGAI, A. Survey on forest fire detection using IoT. Journal of Physics: Conference Series, v. 1911, n. 1, p. 012017, 2021.
- STONEBRAKER, Michael; ÇETINTEMEL, Uğur; ZDONIK, Stanley. The 8 rules of streaming applications. ACM SIGMOD Record, v. 34, n. 4, p. 42-47, 2005.
- TANENBAUM, Andrew S.; VAN STEEN, Maarten. Distributed Systems: Principles and Paradigms. 3. ed. CreateSpace Independent Publishing Platform, 2017.
- TURNER, James et al. Containerization in modern software architectures: A Docker review. IEEE Software, v. 35, n. 2, p. 44-51, 2018.
- ZAHARIA, Matei et al. Apache Spark: A unified engine for big data processing. Communications of the ACM, v. 59, n. 11, p. 56-65, 2016.
- ZASLAVSKY, Arkady; PERERA, Charith; GEORGAKOPOULOS, Dimitrios. Sensing as a service and big data. arXiv preprint arXiv:1301.0159, 2013.
- WHITE, Tom. Hadoop: The Definitive Guide. 4. ed. O'Reilly Media, Inc., 2015.